Single visit based approaches

Analysing Ecological Data with Detection Error

Péter Sólymos

2026-04-20

Table of contents

Preamble 1

Field data analysis 2

Single visit occupancy 3

Single visit N-mixture 8

RSPF condition 13

Preamble

```
suppressPackageStartupMessages({
    library(unmarked)
    library(detect)
    library(dplyr)
    library(ggplot2)
    theme_set(theme_light())
})
set.seed(1234)
```

Field data analysis

We will use the ABMI data, which has 3 equal (3.3 min) time intervals for each point count visit. We use the point level data, but only for the center point, so we can keep sample size small and run the code quickly. When analyzing such data sets for real, we would use all the data and not subset it like this.

```
if (!interactive()) {
    d <- readRDS("../data/ab_birds_site_year_pt.rds")
} else {
    d <- readRDS("data/ab_birds_site_year_pt.rds")
}

spp <- "yellowrumped_warbler"

x <- data.frame(
    y1 = ifelse(d$visits$visit1[, spp] > 0, 1, 0),
    y2 = ifelse(d$visits$visit2[, spp] > 0, 1, 0),
    y3 = ifelse(d$visits$visit3[, spp] > 0, 1, 0),
    n1 = d$visits$visit1[, spp],
    n2 = d$visits$visit2[, spp],
    n3 = d$visits$visit3[, spp],
    d$vars
)
x$yday <- as.POSIXlt(x$fielddate)$yday / 365
x$hr <- x$fieldtime / 24
x$y_max <- pmax(x$y1, x$y2, x$y3)
x$n_max <- pmax(x$n1, x$n2, x$n3)
x <- x |>
    filter(
        hr > 0.1,
        hr < 0.6,
        yday > 0.37,
        yday < 0.5
    )
x0 <- x[x$point == 1, ]
```

Single visit occupancy

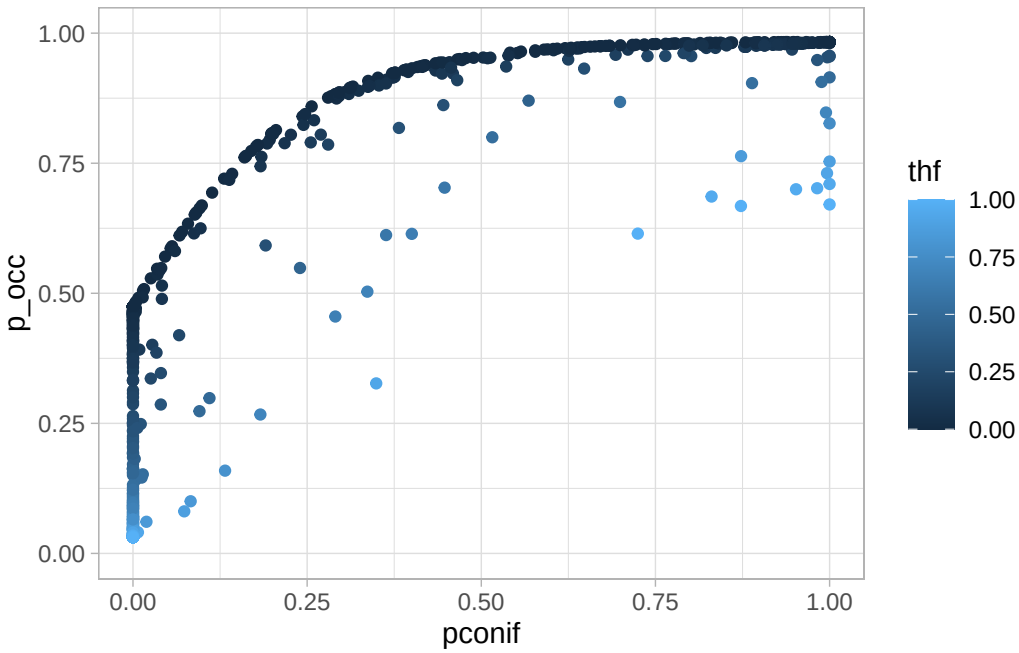
```r
m_sv <- svocc(
    y1 ~ pconif + I(pconif^2) + thf | hr + I(hr^2) + yday,
    data = x0,
    link.sta = "logit"
)
summary(m_sv)
```

```
Call:
svocc(formula = y1 ~ pconif + I(pconif^2) + thf | hr + I(hr^2) + yday,
    data = x0, link.sta = "logit")


Single visit site-occupancy model
Maximum Likelihood estimates (optim method)

Occupancy model coefficients with logit link:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -0.1023     0.3111  -0.329   0.7423
pconif        8.5911     4.9674   1.729   0.0837 .
I(pconif^2)  -4.4403     5.3427  -0.831   0.4059
thf          -3.3370     0.6002  -5.560  2.7e-08 ***
Detection model coefficients with logit link:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)    2.729      2.682   1.017   0.3089
hr             7.566     10.785   0.702   0.4830
I(hr^2)       -8.092     17.133  -0.472   0.6367
yday          -9.511      5.031  -1.890   0.0587 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Log-likelihood: -369.9 on 8 Df
AIC = 755.9
```

```r
fit_sv2 <- plogis(
    model.matrix(formula(m_sv)$sta, x0) %*% coef(m_sv, model = "sta")
)
summary(fit_sv2)
```

```
       V1
 Min.   :0.03109
 1st Qu.:0.05752
 Median :0.47445
 Mean   :0.49834
 3rd Qu.:0.94831
 Max.   :0.98293
```

```
data.frame(x0, p_occ = fit_sv2) |>
    ggplot(aes(x = pconif, y = p_occ, col = thf)) +
    geom_point() +
    ylim(0, 1)
```

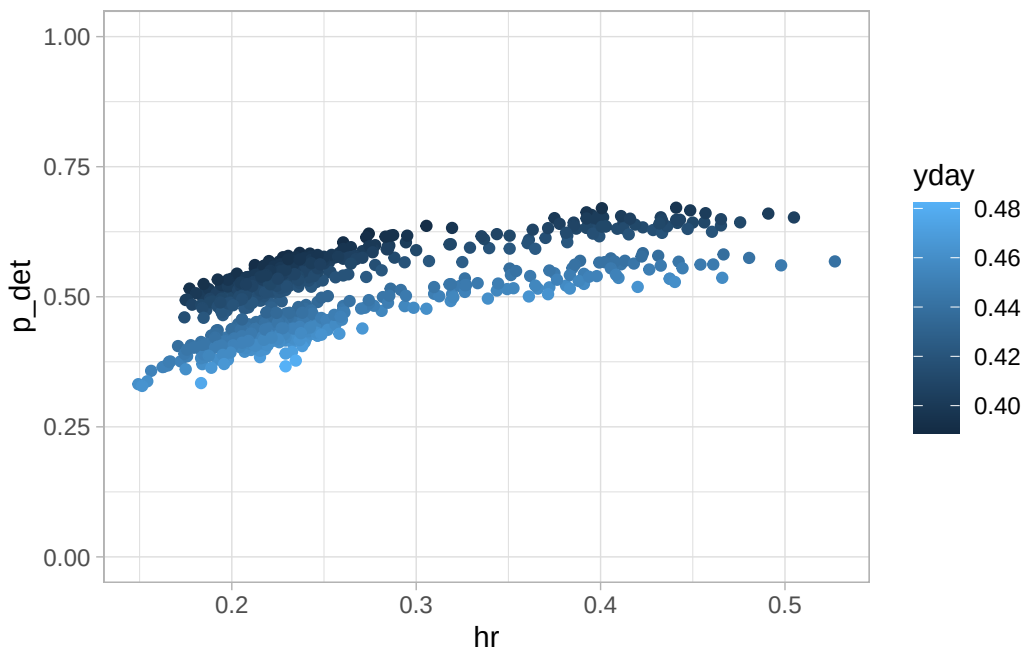



```
d2 <- plogis(
    model.matrix(formula(m_sv)$det, x0) %*% coef(m_sv, model = "det")
)
summary(d2)
```

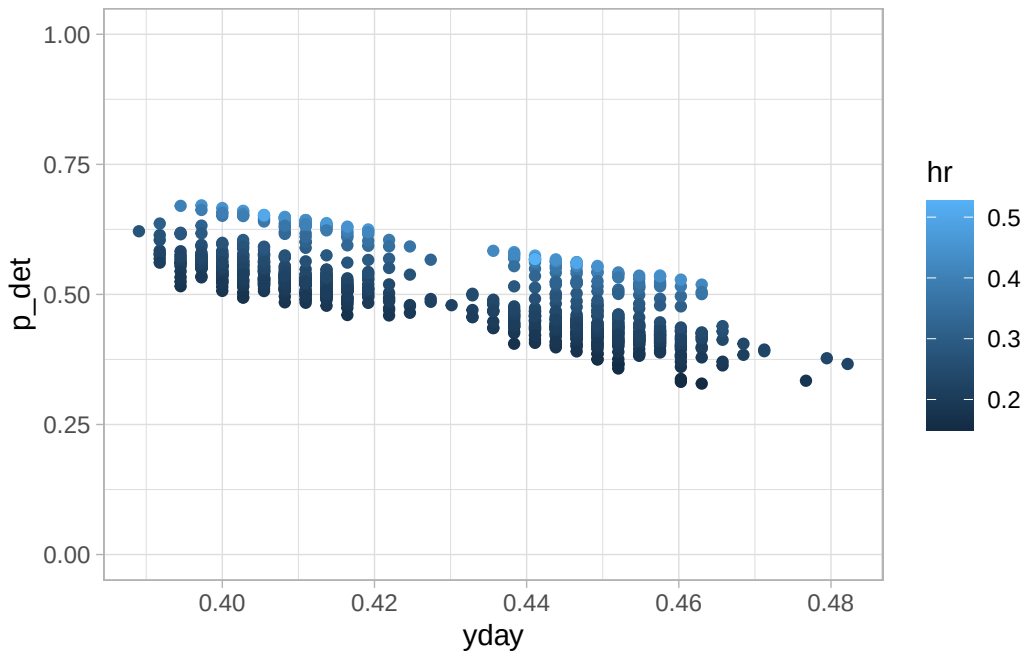
```
       V1
 Min.   :0.3286
 1st Qu.:0.4400
 Median :0.5148
```

```
 Mean   :0.5039
 3rd Qu.:0.5515
 Max.   :0.6712
```

```
data.frame(x0, p_det = d2) |>
    ggplot(aes(x = hr, y = p_det, col = yday)) +
    geom_point() +
    ylim(0, 1)
```




```
data.frame(x0, p_det = d2) |>
    ggplot(aes(x = yday, y = p_det, col = hr)) +
    geom_point() +
    ylim(0, 1)
```




```r
# sometimes it is hard to get SEs, use bootstrap
b <- bootstrap(m_sv, B = 250)
```

```
Warning in svocc.fit(Y, X, Z, link.sta, link.det, penalized = penalized, :
negative variance values in optim, NAs produced
```

```r
summary(b, type = "boot")
```

```
Call:
svocc(formula = y1 ~ pconif + I(pconif^2) + thf | hr + I(hr^2) + yday,
    data = x0, link.sta = "logit")


Single visit site-occupancy model
Maximum Likelihood estimates (optim method)
with Nonparametric Bootstrap standard errors (B = 250)

Occupancy model coefficients with logit link:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -0.1023     0.4509  -0.227    0.821
pconif        8.5911    33.3887   0.257    0.797
```

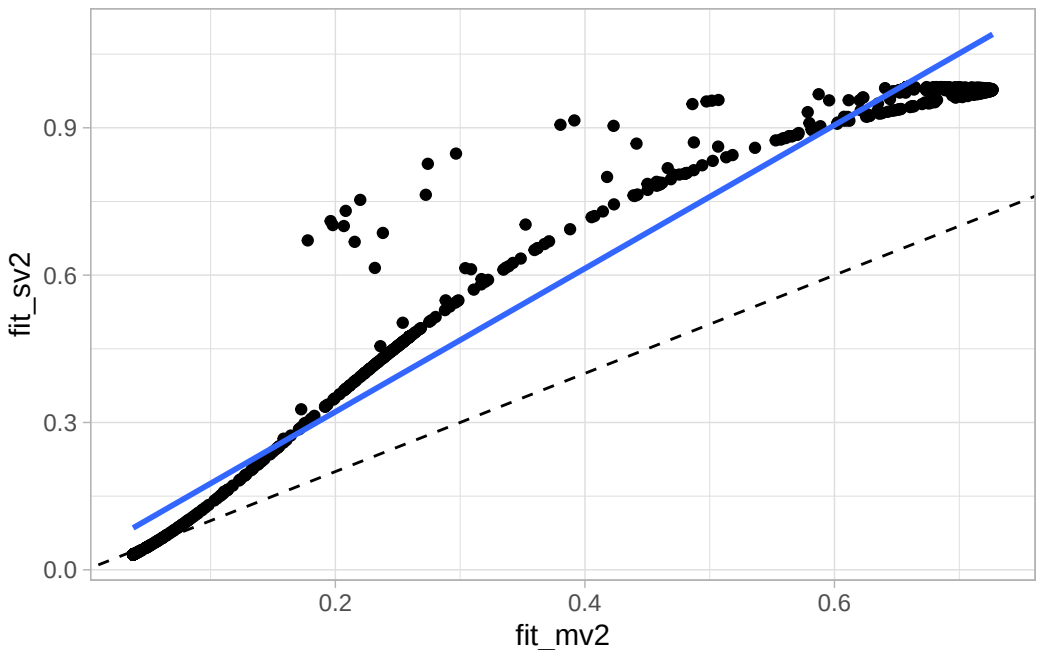
```
I(pconif^2)  -4.4403    26.7084  -0.166    0.868
thf          -3.3370     0.6379  -5.231 1.69e-07 ***
Detection model coefficients with logit link:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)    2.729     11.419   0.239    0.811
hr             7.566     36.451   0.208    0.836
I(hr^2)       -8.092     27.812  -0.291    0.771
yday          -9.511     18.751  -0.507    0.612
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Log-likelihood: -369.9 on 8 Df
AIC = 755.9
```

```r
# explain parallelization options for bootstrap
```

Compare multiple and single visit results:

```r
umf2 <- unmarked::unmarkedFrameOccu(
    y = x0[, c("y1", "y2", "y3")],
    siteCovs = x0[, c("pconif", "thf")],
    obsCovs = list(
        hr = x0[, c("hr", "hr", "hr")],
        yday = x0[, c("yday", "yday", "yday")]
    )
)
m_mv2 <- unmarked::occu(
    ~ hr + I(hr^2) + yday ~ pconif + I(pconif^2) + thf,
    umf2
)
fit_mv2 <- plogis(model.matrix(m_mv2@formula[-2], x0) %*% coef(m_mv2, type = "state"))

data.frame(p_occ = fit_mv2, p_max = fit_sv2) |>
    ggplot(aes(x = fit_mv2, y = fit_sv2)) +
    geom_point() +
    geom_abline(lty = 2) +
    geom_smooth(method = "lm", se = FALSE)
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```
mean(fit_mv2 / fit_sv2)
```

```
[1] 0.7744198
```

Single visit N-mixture

```
m_sv <- svabu(
    n1 ~ pconif + I(pconif^2) + thf | hr + yday,
    data = x0,
    N.max = 100,
    zeroinfl = FALSE
)
summary(m_sv)
```

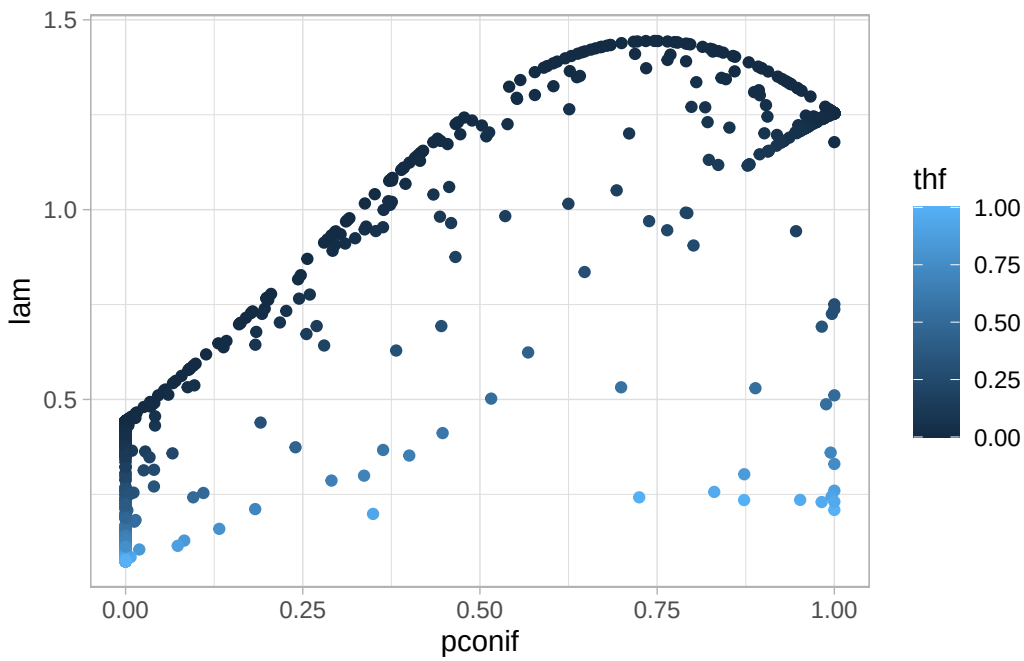
```
Call:
svabu(formula = n1 ~ pconif + I(pconif^2) + thf | hr + yday, data = x0,
    zeroinfl = FALSE, N.max = 100)

Single visit Binomial - Poisson model
```

```
Conditional Maximum Likelihood estimates

Coefficients for abundance (log link):
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -0.8137     0.4631  -1.757 0.078930 .
pconif        3.1805     0.6198   5.132 2.87e-07 ***
I(pconif^2)  -2.1405     0.5992  -3.572 0.000354 ***
thf          -1.7918     0.2836  -6.319 2.63e-10 ***
Coefficients for detection (logit link):
            Estimate Std. Error z value Pr(>|z|)
(Intercept)    8.665      7.847   1.104    0.269
hr             4.361      5.751   0.758    0.448
yday         -21.279     17.872  -1.191    0.234
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Log-likelihood: -579.3 on 7 Df
AIC =  1173
```

```r
fit_mv <- exp(
    model.matrix(formula(m_sv)$sta, x0) %*% coef(m_sv, model = "sta")
)
summary(fit_mv)
```

```
       V1
 Min.   :0.07386
 1st Qu.:0.10420
 Median :0.43309
 Mean   :0.57000
 3rd Qu.:1.11729
 Max.   :1.44443
```

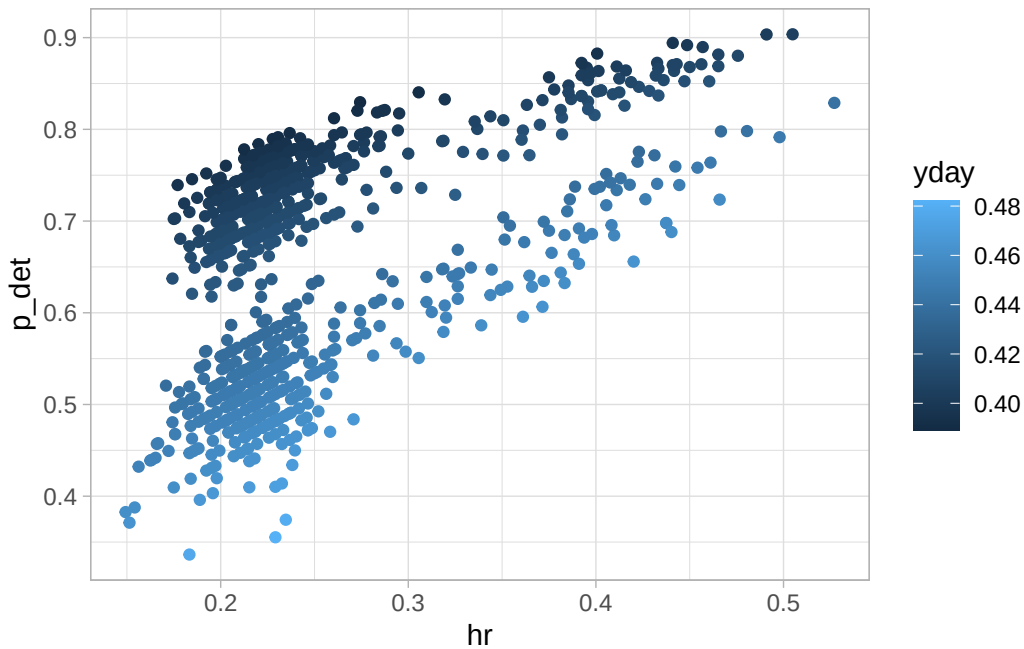
```r
data.frame(x0, lam = fit_mv) |>
    ggplot(aes(x = pconif, y = lam, col = thf)) +
    geom_point()
```



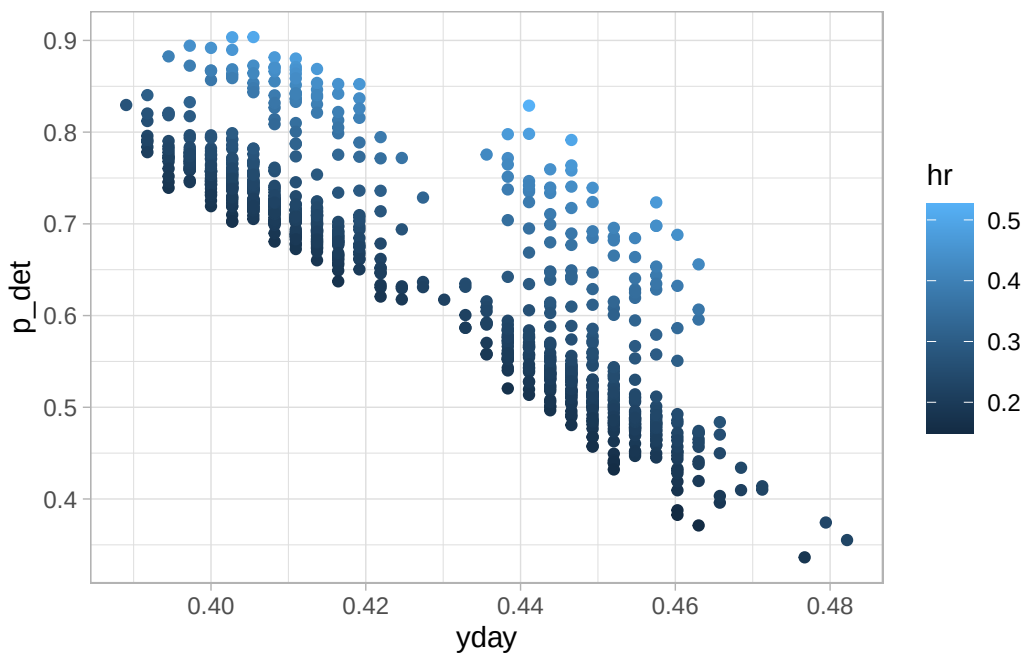

```
d2 <- plogis(
    model.matrix(formula(m_sv)$det, x0) %*% coef(m_sv, model = "det")
)
summary(d2)
```

```
       V1
 Min.   :0.3364
 1st Qu.:0.5366
 Median :0.6858
 Mean   :0.6506
 3rd Qu.:0.7436
 Max.   :0.9037
```

```
data.frame(x0, p_det = d2) |>
    ggplot(aes(x = hr, y = p_det, col = yday)) +
    geom_point()
```




```
data.frame(x0, p_det = d2) |>
    ggplot(aes(x = yday, y = p_det, col = hr)) +
    geom_point()
```



Exercise: Try using `svabu()` with the 2nd and 3rd visits.

What if we stack visits and assume independence?

```
x1 <- x0 |>
    dplyr::select(n1, n2, n3, pconif, thf, hr, yday) |>
    tidyr::pivot_longer(
        cols = starts_with("n"),
        names_to = "visit",
        values_to = "n"
    )
m_sv123 <- svabu(
    n ~ pconif + I(pconif^2) + thf | hr + yday,
    data = x1,
    N.max = 100,
    zeroinfl = FALSE
)
summary(m_sv123)
```

Model selection is more complicated for single visit models, because we need to make sure the model form satisfies the RSPF condition.

The following strategy is recommended for single visit models:

1. Fit the most complex model (saturated) for both the status and detection components
2. Then simplify the model by removing detection covariates one by one, starting with the least significant ones, and checking the AIC at each step
3. Make sure that detection probability has sufficient variation (e.g. by plotting it against covariates), i.e. it is not close to constant, otherwise the model may not be identifiable and estimates may be unreliable
4. Once you have a reasonable detection model, you can simplify the status component by removing covariates one by one, starting with the least significant ones, and checking the AIC at each step

```
m_sv_full <- svabu(
    n1 ~ pconif + I(pconif^2) + thf | hr + I(hr^2) + yday,
    data = x0,
    N.max = 100,
    zeroinfl = FALSE
)
summary(m_sv_full)

drop1(m_sv_full, model = "det") # no huge drop in AIC
m_sv_red <- update(m_sv_full, formula = . ~ . | . - I(hr^2))
```

```r
summary(m_sv_red)
drop1(m_sv_red, model = "sta") # no better model
```

Exercise: Try `zeroinfl = TRUE` and `distr = "NB"` to explore other model forms. Compare the models using AIC.

```r
m_sv_P <- m_sv_red
m_sv_ZIP <- update(m_sv_P, zeroinfl = TRUE)
m_sv_NB <- update(m_sv_P, distr = "NB")
m_sv_ZINB <- update(m_sv_P, distr = "NB", zeroinfl = TRUE)

AIC(m_sv_P, m_sv_NB, m_sv_ZIP, m_sv_ZINB)
summary(m_sv_ZIP)
```

RSPF condition

RSPF condition reduces to the following two conditions:

1. not all covariates in the model are categorical, and
2. the ratio function is nonlinear and involves all components of the parameter vector

Remember that if the nonlinearity on the log-scale is weak, one may need very large sample sizes to get reasonable estimates.

Use the shiny app to explore issues when both the status and detection components of the model are constants:

```r
shiny::runApp("apps/mvisit")
```